

AppEscolar

Sistema de Transporte y Gestion Escolar

Reporte tecnico: Backend, Aplicacion movil y Base de datos

Grupo Exodia

Kotlin + Jetpack Compose | Node.js + Express | Supabase (PostgreSQL)

1. Descripcion del proyecto

AppEscolar es un sistema para el transporte escolar de una institucion educativa. Conecta a los estudiantes (y sus familias) con el conductor de la movilidad, permitiendo seguir el bus en el mapa en tiempo casi real y registrar la entrega de cada estudiante. El sistema tiene tres partes: una aplicacion movil Android, una API REST (backend) y una base de datos en la nube.

La aplicacion maneja **dos roles**:

- **Estudiante:** se registra en la app, fija su punto de recojo con el GPS, ve comunicados, notas y asistencias, sigue el bus en el mapa y guarda contactos de emergencia y recordatorios personales.
- **Conductor:** se precarga en la base (no se registra). Ve la lista de estudiantes de su movilidad, marca a cada uno como entregado o cancelado, y comparte su ubicacion GPS para el seguimiento.

Componentes del sistema

Componente	Tecnologia	Responsabilidad
Aplicacion movil	Kotlin + Jetpack Compose	Interfaz de usuario, GPS, mapa y consumo de la API.
Backend (API REST)	Node.js + Express	Logica de acceso a datos y validaciones. Expone endpoints HTTP.
Base de datos	Supabase (PostgreSQL)	Almacena usuarios, asistencias, ubicacion del bus, comunicados y notas.

2. Arquitectura general

El flujo de una peticion recorre las tres capas del sistema. La app nunca toca la base de datos directamente: siempre pasa por el backend, que es el unico que tiene la clave de Supabase.

```
[ App Android ] --HTTP/JSON--> [ API Express ] --SDK--> [ Supabase / PostgreSQL ]
  ViewModel           Router por           Tablas
  Repository          recurso              (usuarios, ...)
```

Esta separacion aporta seguridad (la credencial de la base vive solo en el servidor), y permite cambiar la interfaz o la base sin reescribir todo.

3. Aplicacion movil

3.1 Stack tecnologico

- **Kotlin y Jetpack Compose** (Material 3) para toda la interfaz.
- **MVVM + Clean Architecture** como patron de organizacion.
- **Retrofit + Gson** para consumir la API (funciones suspend con corrutinas).
- **Google Maps + Fused Location** para el mapa y el GPS.
- **DataStore** (ajustes) y **SQLite** (contactos de emergencia y recordatorios).
- SDK minimo 24, objetivo 36.

3.2 Estructura de carpetas (Clean Architecture)

```
com.example.myapplication/  
|- MainActivity.kt           punto de entrada y navegacion  
|- core/                    sesion, permisos, ajustes en memoria  
|- data/  
|   |- models/              DTOs (JSON de la API)  
|   |- remote/              mapeadores DTO -> entidad de dominio  
|   |- repositories/        unica puerta a los datos  
|   |- local/               SQLite y DataStore  
|- domain/entities/         entidades de negocio (Usuario, Alumno, ...)  
|- services/api/            cliente Retrofit  
|- ui/  
|   |- components/          componentes reutilizables  
|   |- screens/              pantallas (auth, dashboard, profile, secure)  
|   |- theme/               colores y tipografia
```

3.3 Flujo de datos (MVVM)

Cada pantalla sigue siempre el mismo camino, sin saltarse capas:

```
Pantalla (Compose) -> ViewModel -> Repository -> API REST / SQLite
```

- **Pantalla:** solo dibuja y observa el estado (mutableStateOf).
- **ViewModel:** guarda el estado y lanza las operaciones en corrutinas.
- **Repository:** decide de donde vienen los datos (API o local) y devuelve entidades de dominio ya listas para la interfaz.

3.4 Pantallas principales por rol

Rol	Pantalla	Funcion
Ambos	Login / Registro / Recuperar	Inicio de sesion, registro de estudiante y reseteo de contrasena por pregunta de seguridad.
Estudiante	Inicio	Datos del estudiante y accesos a colegio, asistencias y recordatorios.
Estudiante	Seguimiento	Mapa con su punto y el punto del bus; estado de recojo (recogido / aun no).
Estudiante	Asistencias / Notas / Comunicados	Informacion academica leida de la API.
Conductor	Ruta activa	Mapa con los estudiantes y el bus; boton Escanear QR (marca entregado) y Cancelar.

Rol	Pantalla	Funcion
Ambos	Ajustes	Modo oscuro, tamaño de letra y cierre de sesión.

3.5 Mapa, GPS y QR

- El mapa muestra **solo** los puntos fijos de los estudiantes y el punto GPS actual del conductor. No se dibuja la ruta ni se usa la Directions API.
- Al registrarse, el estudiante fija su punto de recojo con el botón 'Usar mi ubicación actual' (GPS del teléfono).
- El botón **Escanear QR** del conductor, por ahora, cambia el estado del estudiante a ENTREGADO. La lectura real del QR con la cámara queda para una etapa posterior.

4. Backend (API REST)

4.1 Stack y estructura

- **Node.js + Express** como servidor HTTP.
- **@supabase/supabase-js** para consultar PostgreSQL.
- Patron **un archivo de rutas por recurso** (router-per-resource).
- Desplegado en **Render**; las credenciales van por variables de entorno.

```
backend-appescolar/  
|- index.js           arranca el servidor  
|- src/  
|   |- app.js         configura Express y engancha las rutas  
|   |- config/supabase.js  conexion a Supabase  
|   |- routes/  
|       |- usuarios.routes.js  
|       |- asistencias.routes.js  
|       |- comunicados.routes.js  
|       |- notas.routes.js  
|       |- ubicaciones.routes.js  
|- db/reset.sql       crea las tablas con datos de ejemplo
```

4.2 Endpoints principales

Metodo	Ruta	Descripcion
POST	/usuarios/regar	Crea la cuenta de un estudiante (con su punto lat/lng).
POST	/usuarios/login	Valida correo y contrasena.
GET	/usuarios/pregunta/:correo	Devuelve la pregunta de seguridad.
POST	/usuarios/restablecer	Cambia la contrasena si la respuesta coincide.
GET	/usuarios/estudiantes/:movilidad	Lista los estudiantes de una movilidad (conductor).
GET	/usuarios/:id	Devuelve un usuario (para consultar su estado).
PUT	/usuarios/:id/estado	Cambia el estado de recojo (PENDIENTE/ENTREGADO/CANCELADO).
GET	/asistencias/:usuariold	Dias con falta, tardanza o justificacion.
GET	/comunicados	Lista de avisos del colegio.
GET	/notas	Lista de calificaciones.
GET	/ubicaciones/:movilidad	Ultima posicion del bus.
PUT	/ubicaciones/:movilidad	El conductor publica su posicion (upsert).

5. Base de datos

La base vive en Supabase (PostgreSQL). Tras la optimizacion del modelo se dejaron **5 tablas**, las unicas que consume la app. El modelo se simplifico para que ambos roles (estudiante y conductor) vivan en una sola tabla **usuarios**.

5.1 Tablas

Tabla	Proposito
usuarios	Cuentas de estudiantes y conductores. Guarda login, datos del estudiante, su punto (lat/Ing), su estado de recojo y el perfil del conductor.
ubicaciones	Una fila por movilidad con la ultima posicion del bus (se sobrescribe).
asistencias	Dias especiales del estudiante (falta, tardanza, justificado).
comunicados	Avisos del colegio.
notas	Calificaciones del estudiante.

5.2 Tabla usuarios (columnas clave)

Columna	Tipo	Descripcion
id	bigint	Identidad. Clave primaria.
correo / contraseña	text	Credenciales de acceso (correo unico).
pregunta / respuesta	text	Pregunta de seguridad para recuperar la contraseña.
rol	text	ESTUDIANTE o CONDUCTOR.
estudiante_nombre / grado	text	Datos del estudiante asociado a la cuenta.
movilidad	text	Bus al que pertenece (ej. Movilidad N04).
lat / Ing	double	Punto de recojo del estudiante.
estado	text	Recojo: PENDIENTE / ENTREGADO / CANCELADO.
celular, dni, licencia, placa, zona	text	Perfil del conductor (solo rol CONDUCTOR).

5.3 Modelo logico

La relacion es simple y por valor (no por llave foranea rigida), para mantener el backend ligero:

- Un **conductor** y varios **estudiantes** se relacionan por el campo **movilidad** (mismo texto).
- Cada fila de **ubicaciones** se une a los usuarios por **movilidad**.
- Cada fila de **asistencias** apunta al estudiante por **usuario_id**.

6. Seguridad y configuracion

- La clave de **Supabase** se usa solo en el servidor (variables de entorno en Render), nunca en la app.
- La clave de **Google Maps** vive en **local.properties** (fuera del control de versiones).

- La recuperacion de contrasena no muestra la clave: se **restablece** respondiendo la pregunta de seguridad.

Despliegue

- **Backend:** Render (Build: npm install, Start: npm start). Entrega una URL publica que la app usa como servidor.
- **Base de datos:** Supabase; el esquema y los datos de ejemplo se crean con db/reset.sql.
- **App:** se compila en Android Studio apuntando a la URL del backend.